Hashing และ Hash-based Data Structures

Data Structures & Object-Oriented Programming



ปัญหาของ Data Structures ที่เรียนมา

Data Structure	Search	Insert	Delete
Linked List	O(n)	O(1) ที่ head	O(n)
Sorted Array	O(log n) ด้วย Binary Search	O(n)	O(n)
Stack / Queue	ไม่รองรับการค้นหาโดยตรง	O(1)	O(1)

- ไม่มีโครงสร้างข้อมูลใดในตารางนี้ที่ทำได้ทั้ง search, insert และ delete ในเวลา O(1) พร้อมกันทั้งหมด
  - Sorted Array ทำ search ได้เร็วด้วย binary search แต่ insert และ delete ช้าเพราะต้องเลื่อน element
  - Linked List ทำ insert ที่ head ได้เร็ว แต่ search กลับต้อง traverse ทีละตัว
- สมมติว่าระบบสมัครสมาชิกออนไลน์ต้องตรวจสอบว่า username ที่ผู้ใช้กรอกเข้ามาซ้ำกับที่มีอยู่แล้วหรือไม่ โดยมีผู้ใช้ลงทะเบียนอยู่แล้วถึง 10 ล้านคน
  - หากเก็บ username ทั้งหมดไว้ใน sorted array และใช้ binary search การตรวจสอบแต่ละครั้งจะใช้เพียงประมาณ 23 ขั้นตอนเท่านั้น (คำนวณจาก $\log_2(10{,}000{,}000) \approx 23.25$) ซึ่งถือว่าเร็วมากอยู่แล้ว
  - แต่หากต้องการให้เป็น O(1) ไม่ว่าจำนวนผู้ใช้จะเพิ่มขึ้นเป็นเท่าใดก็ตาม Hash Table คือคำตอบของปัญหานี้

แนวคิดหลัก: แปลง Key เป็น Index โดยตรง

- หัวใจสำคัญของ Hash Table คือ มันใช้สิ่งที่เรียกว่า hash function เพื่อคำนวณตำแหน่ง (index) ของ key นั้นโดยตรงในทันที แทนที่จะค้นหาตำแหน่งของ key ทีละขั้นตอน

```
key = "alice"
index = hash("alice") % capacity      → ได้ผลลัพธ์ เช่น 3
arr[3] = value                         → เข้าถึงได้ทันที O(1)
```

- เนื่องจากการเข้าถึง array ด้วย index เป็น O(1) เสมอ
- หากสามารถคำนวณ index จาก key ได้โดยตรง ก็ใช้เวลา O(1) เช่นกัน
- ผลลัพธ์โดยรวมของการเข้าถึงข้อมูลด้วย key จึงกลายเป็น O(1) ไปด้วย
- ดังนั้น Hash Table จึงมีความเร็วในการเข้าถึงข้อมูลที่โดดเด่น

Hash Function

3 คุณสมบัติของ Hash Function

Hash function ที่ดีต้องมีคุณสมบัติสำคัญ 3 ประการพร้อมกัน หากขาดคุณสมบัติใดไปเพียงข้อเดียว Hash Table ทั้งระบบจะทำงานผิดพลาดหรือด้อยประสิทธิภาพลงอย่างมาก

- Deterministic (ให้ผลลัพธ์คงที่) หมายถึง key เดิมต้องให้ค่า index เดิมทุกครั้งที่คำนวณ ไม่ว่าจะเรียกกี่ครั้งก็ตาม
  - หาก hash("alice") ให้ผลลัพธ์เป็น 3 ในการคำนวณครั้งแรก ครั้งถัดไปก็ต้องได้ 3 เช่นเดิมเสมอ
  - หากขาดคุณสมบัตินี้: การค้นหาข้อมูลที่เคยเก็บไว้จะไม่มีทางสำเร็จ เพราะระบบจะค้นหาผิดตำแหน่งเสมอ
- Uniform (กระจายสม่ำเสมอ) หมายถึง hash function ควรกระจาย key ต่างๆ ไปยังตำแหน่งต่างๆ ของ array อย่างสม่ำเสมอ ไม่ให้กระจุกตัวอยู่ที่ตำแหน่งใดตำแหน่งหนึ่งมากเกินไป
  - หากขาดคุณสมบัตินี้: บาง bucket จะมีจำนวน key สะสมอยู่มากผิดปกติ (เรียกว่า clustering) ในขณะที่ bucket อื่นว่างเปล่า ทำให้ความซับซ้อนของ operation ในบาง bucket นั้นเสื่อมถอยจาก O(1) ไปเป็น O(n) ได้ในทางปฏิบัติ แม้ในทางทฤษฎีจะยังคงเรียกว่า Hash Table อยู่ก็ตาม

3 คุณสมบัติของ Hash Function

Hash function ที่ดีต้องมีคุณสมบัติสำคัญ 3 ประการพร้อมกัน หากขาดคุณสมบัติใดไปเพียงข้อเดียว Hash Table ทั้งระบบจะทำงานผิดพลาดหรือด้อยประสิทธิภาพลงอย่างมาก

- Fast (คำนวณได้รวดเร็ว) หมายถึง การคำนวณ hash function เองต้องใช้เวลา O(1)
  - หากขาดคุณสมบัตินี้: เช่น ต้องประมวลผลข้อมูลที่ซับซ้อนมากก่อนจะได้ index ต้นทุนของการคำนวณ hash จะไปกลืนกินข้อได้เปรียบด้าน O(1) ที่ Hash Table ควรจะทำได้

การใช้งาน hash() ใน Python

- Python มีฟังก์ชัน built-in ชื่อ hash() ที่รองรับข้อมูลหลายชนิด ทั้ง string, int, tuple และ object ต่างๆ

```
print(hash("alice") % 8) # อาจได้ 3 (ขึ้นอยู่กับ Python version และ hash seed)
print(hash(42) % 8)      # 2  ← เพราะ hash(42) == 42 สำหรับ integer และ 42 % 8 = 2
print(hash((1, 2)) % 8)  # ค่าใดค่าหนึ่ง ขึ้นอยู่กับการคำนวณ hash ของ tuple
```

- Python สุ่มค่าเริ่มต้น (hash seed) สำหรับการคำนวณ hash ของ string ใหม่ทุกครั้งที่เริ่มต้น interpreter
  - เพื่อป้องกันการโจมตีที่เรียกว่า hash flooding (ผู้โจมตีจงใจส่ง key จำนวนมากที่คำนวณ hash ชนกันเพื่อทำให้ระบบช้าลงอย่างรุนแรง หากรู้ล่วงหน้าว่า hash function คำนวณอย่างไร)
  - ด้วยเหตุนี้ ค่า hash("alice") จึงอาจแตกต่างกันระหว่างการรันโปรแกรมคนละครั้ง แต่ภายในการรันครั้งเดียวกัน ค่านี้จะคงที่เสมอ ไม่ขัดกับคุณสมบัติ deterministic ที่กล่าวไว้

สูตรพื้นฐานในการคำนวณ Bucket Index

index = hash(key) % capacity

- การใช้เครื่องหมาย modulo (%) กับค่า capacity
  - ค่า hash(key) ที่ได้จากฟังก์ชัน hash() อาจเป็นจำนวนเต็มขนาดใหญ่มาก (หรือติดลบ) ซึ่งเกินขอบเขตของ array ที่มีขนาดจำกัด
  - การหารเอาเศษด้วย capacity จะบังคับให้ผลลัพธ์อยู่ในช่วง 0 ถึง capacity - 1 เสมอ ซึ่งเป็นช่วง index ที่ใช้งานได้จริงของ array นั้น

Collision

Collision คืออะไร

- เหตุการณ์ที่ key สองตัวซึ่งมีค่าแตกต่างกันโดยสิ้นเชิง กลับคำนวณ hash index ได้ตรงกันพอดี
- ตัวอย่างเช่น
  - hash("alice") % 8 == hash("charlie") % 8 == 3
  - แม้ "alice" และ "charlie" จะเป็นคนละ string กันอย่างชัดเจน แต่เมื่อผ่านการคำนวณ hash แล้วหารเอาเศษด้วย capacity อาจได้ผลลัพธ์เป็นตัวเลขเดียวกันได้

ทำไม Collision จึงหลีกเลี่ยงไม่ได้โดยหลักการ

- หลักการทางคณิตศาสตร์ที่เรียกว่า Pigeonhole Principle (หลักรังนกพิราบ)
  - หากมีจำนวนสิ่งของ (items) มากกว่าจำนวนช่องที่จะใส่ (buckets) จะต้องมีอย่างน้อยหนึ่งช่องที่มีสิ่งของมากกว่าหนึ่งชิ้นอยู่เสมอ อย่างหลีกเลี่ยงไม่ได้
  - ในบริบทของ Hash Table หากมีจำนวน key ที่เป็นไปได้ (n) มากกว่าจำนวน bucket ที่มีอยู่ (m) การเกิด collision จึงเป็นสิ่งที่รับประกันได้ว่าจะเกิดขึ้นแน่นอนในบางจุด
- แม้ในกรณีที่ n น้อยกว่า m มาก collision ก็ยังคงเกิดขึ้นได้ง่ายกว่าที่คาดคิดไว้มาก ปรากฏการณ์นี้อธิบายได้ด้วยหลักการที่เรียกว่า Birthday Problem (ปัญหาวันเกิด)
  - ในห้องที่มีคนอยู่เพียง 23 คน มีโอกาสมากกว่า 50% ที่จะมีคนสองคนเกิดวันเดียวกัน (เดือนและวันตรงกัน) ทั้งที่จำนวนวันในหนึ่งปีมีถึง 365 วัน ซึ่งมากกว่าจำนวนคนในห้องถึง 15 เท่า
  - เหตุผลเบื้องหลังคือ การเปรียบเทียบไม่ได้เกิดขึ้นแค่ระหว่างคนสองคนที่กำหนดไว้ล่วงหน้า แต่เกิดขึ้นระหว่างคู่ที่เป็นไปได้ทั้งหมดในกลุ่ม ซึ่งเพิ่มขึ้นอย่างรวดเร็วตามจำนวนคน (สำหรับ 23 คน มีคู่ที่เป็นไปได้ถึง 23×22/2 = 253 คู่)
- สำหรับ Hash Table แม้จำนวน key ที่ใส่เข้าไปจะน้อยกว่าจำนวน bucket มาก แต่โอกาสที่จะเกิด collision สักครั้งหนึ่งก็ยังคงสูงกว่าที่คาดคิดไว้เสมอ จึงจำเป็นต้องมีกลไกจัดการ collision รองรับไว้เสมอ

Collision Resolution: Chaining

- Chaining แก้ปัญหา collision ด้วยแนวคิดที่เรียบง่ายที่สุดเท่าที่จะเป็นไปได้
- แทนที่จะให้แต่ละ bucket เก็บได้เพียงหนึ่ง entry เท่านั้น ให้แต่ละ bucket เป็น list ของ (key, value) pairs
  - เมื่อเกิด collision ขึ้น (มี key ใหม่คำนวณ index ได้ตรงกับ bucket ที่มีข้อมูลอยู่แล้ว) เพียงแค่ append entry ใหม่เข้าไปต่อท้าย list ของ bucket นั้น โดยไม่ต้องทำอะไรพิเศษเพิ่มเติม
- ลองพิจารณาตัวอย่าง Hash Table ที่มี capacity เท่ากับ 8 และทำการ insert "alice", "bob", "charlie" โดยสมมติว่า hash("alice") % 8 = 3, hash("bob") % 8 = 6, hash("charlie") % 8 = 3 (ชนกับ alice พอดี)

```
[0] → []
[1] → []
[2] → []
[3] → [("alice", val1), ("charlie", val3)]
[4] → []
[5] → []
[6] → [("bob", val2)]
[7] → []
```

- "alice" และ "charlie" ซึ่งชนกันที่ bucket 3 ต่างก็ยังคงอยู่ได้ในระบบ เพียงแต่ถูกเก็บรวมกันไว้ใน list เดียวกันของ bucket นั้น
- เมื่อต้องการค้นหา "charlie" ในภายหลัง ระบบจะคำนวณ index ได้เป็น 3 เช่นเดิม แล้วไล่ตรวจดู list ที่ bucket 3 ทีละ entry จนกว่าจะพบ key ที่ตรงกัน

ความซับซ้อนของ Chaining

- ความซับซ้อนของ operation ต่างๆ ใน Chaining ขึ้นอยู่กับความยาวเฉลี่ยของแต่ละ chain (คือความยาวของ list ในแต่ละ bucket) โดยตรง
  - หาก hash function กระจาย key ได้ดีตามคุณสมบัติ uniform และ load factor อยู่ในระดับต่ำ แต่ละ chain จะมีความยาวใกล้เคียงกับ 1 เสมอ ทำให้การไล่ตรวจ list ใน bucket ใดๆ ใช้เวลาแทบจะคงที่
  - ส่งผลให้ insert, search และ delete ทั้งหมดมีความซับซ้อนเป็น O(1) โดยเฉลี่ย (average case)
- แต่ในกรณีที่เลวร้ายที่สุด (worst case) หาก hash function ทำงานได้แย่มาก
  - ทุก key ที่ใส่เข้ามาชนกันที่ bucket เดียวกันหมด chain ของ bucket นั้นจะมีความยาวเท่ากับ n (จำนวน key ทั้งหมด)
  - ทำให้การไล่ตรวจ list กลายเป็น O(n) worst case ซึ่งไม่ต่างอะไรจากการค้นหาแบบ linear search ใน Linked List ธรรมดาเลย

Collision Resolution: Linear Probing

- Linear Probing เก็บข้อมูลทั้งหมดไว้ใน array เดียว และเมื่อเกิด collision ที่ตำแหน่งใด ให้ “probe” (สำรวจ) หาตำแหน่งว่างถัดไปในarray ไปเรื่อยๆ ทีละช่อง จนกว่าจะพบตำแหน่งว่าง

```
insert "alice"   → index 3: arr[3] ว่าง          → วางที่ 3
insert "charlie" → index 3: arr[3] ไม่ว่าง (มี alice) → ลอง index 4: ว่าง → วางที่ 4
insert "dave"    → index 4: arr[4] ไม่ว่าง (มี charlie) → ลอง index 5: ว่าง → วางที่ 5
```

- แนวทางนี้เหมาะสมกับสถานการณ์ที่ต้องการให้ข้อมูลอยู่ใน memory ที่กระชับ (compact) มากกว่า Chaining เพราะไม่ต้องมี pointer เพิ่มเติมสำหรับ linked list
  - ข้อมูลทั้งหมดอยู่ใน array ต่อเนื่องกันในหน่วยความจำ ซึ่งยังส่งผลดีต่อ cache performance

ปัญหาสำคัญของการ Delete ใน Linear Probing

- Linear Probing มีปัญหาที่ซับซ้อนกว่า Chaining อย่างมากเมื่อต้องทำการ delete
- จากตัวอย่าง สมมติว่าต้องการ ลบ "alice" ที่ index 3 ออกไป โดยใส่ None กลับเข้าไปที่ตำแหน่งนั้น

```
[3] → None        (เดิมคือ "alice", ถูกลบแล้ว)
[4] → "charlie"
[5] → "dave"
```

- ปัญหาจะปรากฏขึ้นทันทีเมื่อพยายาม get("charlie") ในภายหลัง ระบบจะคำนวณ index ของ "charlie" ได้เป็น 3 (ตำแหน่งเดิมที่มันเคยชนกับ alice) แล้วเริ่มตรวจสอบที่ index 3 ก่อน พบว่าเป็น None
  - ตามหลักการ probing ทั่วไป None หมายถึง “ไม่เคยมี key ใดถูกวางไว้ที่นี่เลย จึงไม่มีเหตุผลที่จะ probe ต่อไปข้างหน้าอีก” ระบบจึงหยุดการค้นหาทันทีและสรุปว่าไม่พบ "charlie" เกิด KeyError
  - ทั้งที่ในความเป็นจริง "charlie" ยังคงอยู่ที่ index 4 อย่างสมบูรณ์

ทางแก้: Tombstone

- วิธีแก้ปัญหานี้คือการใช้ Tombstone ซึ่งเป็น marker พิเศษที่มีความหมายแตกต่างจาก None โดยสิ้นเชิง
  - Tombstone สื่อความหมายว่า “เคยมี key อยู่ที่ตำแหน่งนี้มาก่อน แต่ถูกลบไปแล้ว ให้ probe ต่อไปข้างหน้าเรื่อยๆ เพราะอาจมี key อื่นที่เคยชนกันแล้วถูก probe ไปไกลกว่านี้

```
TOMBSTONE = object()   # sentinel object ที่ไม่ซ้ำกับค่าใดๆ ในโลก

# delete: ใส่ TOMBSTONE แทน None
arr[idx] = TOMBSTONE

# search: ข้าม TOMBSTONE แต่อย่าหยุดค้นหา (ต่างจาก None ที่ต้องหยุดทันที)
while arr[idx] is not None:
    if arr[idx] is not TOMBSTONE and arr[idx][0] == key:
        return arr[idx][1]
    idx = (idx + 1) % capacity
```

- เงื่อนไขการหยุด loop ยังคงเป็น arr[idx] is not None เหมือนเดิม (Tombstone ไม่ทำให้ loop หยุด) แต่มีการเพิ่มเงื่อนไข arr[idx] is not TOMBSTONE เข้ามาก่อนตรวจสอบว่า key ตรงกันหรือไม่
- Tombstone ไม่ใช่ entry ที่ถูกต้อง จึงไม่มีทางที่ arr[idx][0] == key จะเป็นจริงได้ในกรณีนี้อยู่แล้ว แต่การเขียนเงื่อนไขแยกไว้ชัดเจนช่วยให้โค้ดสื่อความหมายได้ตรงและป้องกันข้อผิดพลาดที่อาจเกิดจากการพยายามเข้าถึง arr[idx][0] ของ object ที่ไม่ใช่ tuple

- การใช้ is แทน == ในการเปรียบเทียบกับ TOMBSTONE เนื่องจาก TOMBSTONE = object()
- สร้าง object เปล่าที่ไม่ซ้ำกับค่าใดๆ ในโลกเลย การเปรียบเทียบด้วย is (ตรวจสอบว่าเป็น object เดียวกันในหน่วยความจำหรือไม่) จึงแม่นยำและปลอดภัยกว่าการใช้ == ซึ่งอาจถูก override พฤติกรรมได้หากมีการ implement __eq__ ในบาง class โดยไม่ตั้งใจ

Load Factor

Load Factor คืออะไร

- คือ อัตราส่วนระหว่างจำนวน entry ที่มีอยู่จริงในตาราง กับความจุ (capacity) ทั้งหมดของตาราง
  - load_factor = size / capacity
  - ค่านี้บอกว่าตารางกำลัง “แออัด” มากน้อยเพียงใด
- เมื่อ load factor มีค่าสูงขึ้นเรื่อยๆ (เข้าใกล้ 1.0) หมายความว่าจำนวน entry ใกล้เคียงกับจำนวน bucket ที่มีอยู่มากขึ้น
  - ยิ่งอัตราส่วนนี้สูง โอกาสเกิด collision ก็ยิ่งสูงตามไปด้วย ไม่ว่าจะใช้ Chaining หรือ Linear Probing ก็ตาม
    - chain จะยาวขึ้นเรื่อยๆ (สำหรับ Chaining)
    - ต้อง probe ไกลขึ้นเรื่อยๆ (สำหรับ Linear Probing)
  - ทำให้ประสิทธิภาพที่แท้จริงในทางปฏิบัติเข้าใกล้ O(n) มากขึ้นเรื่อยๆ แม้ในทางทฤษฎีจะยังเรียกว่า “O(1) average”
- เกณฑ์มาตรฐานที่นิยมใช้กันคือ **0.75**
  - เมื่อ load factor เกิน 0.75 (มี entry มากกว่า 3 ใน 4 ของความจุทั้งหมด) ระบบจะดำเนินการที่เรียกว่า **rehash** ทันที เพื่อรักษาประสิทธิภาพของ Hash Table ไว้ไม่ให้เสื่อมถอยลงมากเกินไป

Rehashing คืออะไร

- กระบวนการสร้าง array (หรือ bucket list) ขึ้นใหม่ที่มีขนาดใหญ่ขึ้น (โดยทั่วไปคือขยายเป็น 2 เท่า ของขนาดเดิม)
  - แล้วนำ entry ทั้งหมดจาก array เดิมมา insert ใหม่ทั้งหมด ลงใน array ใหม่นี้

```
def _rehash(self):
    old_buckets = self._buckets
    self._capacity *= 2                                  # ขยาย 2 เท่า
    self._buckets = [[] for _ in range(self._capacity)]
    self._size = 0
    for bucket in old_buckets:                           # re-insert ทุก entry
        for key, value in bucket:
            self.insert(key, value)
```

- ทำไมต้อง insert ใหม่ทั้งหมด ไม่แค่ย้าย entry ไปยัง array ใหม่ตรงๆ
  - เพราะสูตรการคำนวณ index = hash(key) % capacity
  - เมื่อค่า capacity เปลี่ยนแปลงไป (เช่นจาก 4 เป็น 8) ผลลัพธ์ของการหารเอาเศษ (%) ย่อมเปลี่ยนแปลงไปด้วย
  - ตำแหน่งที่ key แต่ละตัวเคยอยู่ใน array เดิมจึงไม่ถูกต้องอีกต่อไปในบริบทของ array ใหม่ที่มี capacity ต่างออกไป จำเป็นต้องคำนวณ index ใหม่ทั้งหมดให้ตรงกับ capacity ใหม่เสมอ

Amortized Cost ของ Rehashing

- แม้ rehashing แต่ละครั้งจะมีความซับซ้อนสูงถึง O(n) (เพราะต้อง insert ใหม่ทุก entry)
- แต่เหตุการณ์นี้เกิดขึ้นไม่บ่อยนัก เพราะหลังจาก rehash แล้ว capacity จะเพิ่มขึ้นเป็นสองเท่าทันที ทำให้ต้อง insert อีกจำนวนมากพอสมควรกว่าที่ load factor จะกลับไปแตะ 0.75 อีกครั้ง
- ค่าใช้จ่ายที่สูงของการ rehash หนึ่งครั้งจะถูก “เฉลี่ย” กระจายออกไปบน insert จำนวนมากที่เกิดขึ้นระหว่างสอง rehash ทำให้เมื่อคำนวณโดยเฉลี่ยแล้ว แต่ละ insert ยังคงมีความซับซ้อนเป็น O(1) amortized อยู่เช่นเดิม

Python dict

Python dict

- โครงสร้างข้อมูล dict ที่ใช้งานกันอยู่ทั่วไปในภาษา Python นั้น คือ Hash Table ที่ implement ไว้ภายในตัวภาษา
- มีลักษณะสำคัญ 3 ประการ
  - 1. รับประกันลำดับการใส่ข้อมูล (insertion order)
    - การวน loop บน dict จะได้ key เรียงตามลำดับที่ถูกใส่เข้าไปจริง
      - ไม่ใช่เรียงตามค่า hash หรือตำแหน่ง bucket
      - คุณสมบัตินี้อาจดูขัดกับสิ่งที่เรียนมา (ที่ entry กระจัดกระจายไปตามค่า hash)
      - ในความเป็นจริง Python เก็บข้อมูลเพิ่มเติมเกี่ยวกับลำดับการใส่ไว้แยกต่างหาก เพื่อมอบคุณสมบัตินี้ให้ผู้ใช้งานโดยไม่กระทบกับความเร็วของการค้นหา

```
d2 = {}
for word in ["banana", "apple", "cherry", "date"]:
    d2[word] = len(word)
print(list(d2.keys()))   # ['banana', 'apple', 'cherry', 'date'] -- ลำดับเดิมเป๊ะ
```

Python dict

- 2. ใช้ compact table
  - Python dict แยกเก็บ array ขนาดเล็กสำหรับ index/hash ออกจาก array ของ entry จริง ทำให้ประหยัดหน่วยความจำมากกว่า implementation แบบตรงไปตรงมาที่เรียน (รายละเอียดอยู่นอกขอบเขตรายวิชา)
- 3. key ต้อง hashable
  - เงื่อนไขสำคัญที่สุดของการใช้ค่าใดๆ เป็น key ใน dict คือค่านั้นต้อง hashable และที่สำคัญที่สุดคือต้องไม่ใช่ object ที่เปลี่ยนแปลงค่าได้ (mutable)

```
d = {}
d[42] = "int key"           # ✓ int เป็น hashable
d["hello"] = "str key"      # ✓ str เป็น hashable
d[(1, 2)] = "tuple key"     # ✓ tuple เป็น hashable (ถ้า element ข้างในก็ hashable ด้วย)

try:
    d[[1, 2]] = "list key"
except TypeError as e:
    print(e)   # unhashable type: 'list'
```

Python dict

- เหตุผลที่ list ไม่ hashable เพราะ list เป็น mutable object สามารถแก้ไขค่าภายในได้หลังจากสร้างขึ้นมาแล้ว
- หากยอมให้ list เป็น key ได้ และมีการคำนวณ hash([1, 2]) ไปแล้ว เก็บไว้ที่ bucket หนึ่ง จากนั้นมีการแก้ไข list ตัวเดิมนั้นเป็น [1, 2, 3]
  - ค่า hash ที่คำนวณได้จะเปลี่ยนไปจากเดิม แต่ตำแหน่งที่เก็บไว้ใน Hash Table ยังคงเป็นตำแหน่งเดิม (ตามค่า hash เก่า)
  - ทำให้ไม่สามารถค้นหา entry นั้นเจอได้อีกเลยในอนาคต
- Python จึงป้องกันปัญหานี้ตั้งแต่ต้นทาง ด้วยการห้ามใช้ mutable object เป็น key โดยสิ้นเชิง

Code ของ HashTable (Chaining)

HashTable (Chaining)

```python
class HashTable:
    """
    Hash Table ด้วย Chaining เป็น collision resolution strategy
    Average O(1) insert/get/delete | Worst O(n) เมื่อทุก key ชนกัน
    """

    def __init__(self, initial_capacity: int = 8):
        self._capacity = initial_capacity
        self._size = 0
        self._buckets = [[] for _ in range(self._capacity)]

    def _hash(self, key) -> int:
        """คำนวณ bucket index จาก key -- O(1)"""
        return hash(key) % self._capacity

    def _load_factor(self) -> float:
        """คืนค่า load factor ปัจจุบัน"""
        return self._size / self._capacity
```

- __init__ ตั้งค่า _capacity ตามที่ผู้ใช้กำหนด (default 8) และสร้าง _buckets เป็น list ของ list ว่างเปล่าจำนวนเท่ากับ capacity โดยใช้ list comprehension [[] for _ in range(self._capacity)]
- ข้อควรระวัง คือ ห้ามเขียนเป็น [[]] * capacity เพราะวิธีนั้นจะสร้าง list ว่างเพียงหนึ่งตัวแล้วอ้างอิงซ้ำ (reference เดียวกัน) ไปยังทุกตำแหน่ง
- ทำให้เมื่อแก้ไข bucket หนึ่งตำแหน่ง bucket ทั้งหมดจะเปลี่ยนตามไปด้วยโดยไม่ตั้งใจ

_hash เรียกใช้ hash() built-in ของ Python แล้วหารเอาเศษด้วย _capacity ตามสูตร

HashTable (Chaining)

```
    def insert(self, key, value) -> None:
        """เพิ่มหรืออัปเดต (key, value) -- amortized O(1)"""
        if self._load_factor() >= 0.75:          # ตรวจสอบและ rehash ก่อนเสมอ
            self._rehash()
        idx = self._hash(key)
        bucket = self._buckets[idx]
        for i, (k, v) in enumerate(bucket):
            if k == key:                         # key มีอยู่แล้ว -> อัปเดต
                bucket[i] = (key, value)
                return
        bucket.append((key, value))              # key ใหม่ -> append
        self._size += 1
```

insert ต้องรับผิดชอบถึงสามเรื่องพร้อมกัน คือ

- (1) ตรวจสอบและทำ rehash ก่อนเสมอหาก load factor ถึงเกณฑ์
- (2) ตรวจสอบว่า key นี้มีอยู่แล้วในระบบหรือไม่ หากมีให้อัปเดต value แทนที่จะเพิ่มซ้ำ และ
- (3) หากเป็น key ใหม่จริงๆ จึง append เข้า bucket แล้วเพิ่ม _size
- การเรียงลำดับของสามขั้นตอนนี้สำคัญมาก โดยเฉพาะจุดที่ 1 ต้องทำก่อนการคำนวณ idx เสมอ

HashTable (Chaining)

```python
def get(self, key):
    """ค้นหา value จาก key -- average O(1). Raises: KeyError ถ้าไม่พบ"""
    idx = self._hash(key)
    for k, v in self._buckets[idx]:
        if k == key:
            return v
    raise KeyError(f"Key not found: {key!r}")

def delete(self, key) -> None:
    """ลบ entry ที่มี key นั้น -- average O(1). Raises: KeyError ถ้าไม่พบ"""
    idx = self._hash(key)
    bucket = self._buckets[idx]
    for i, (k, v) in enumerate(bucket):
        if k == key:
            bucket.pop(i)
            self._size -= 1
            return
    raise KeyError(f"Key not found: {key!r}")
```

- get และ delete มีโครงสร้างคล้ายกันมาก คือคำนวณ idx แล้วไล่ตรวจ bucket ที่ตำแหน่งนั้นทีละ entry จนกว่าจะพบ key ที่ตรงกัน
- หากไม่พบเลยจนครบ bucket ทั้งหมด (คือ loop จบโดยไม่มีการ return) จะ raise KeyError พร้อมข้อความที่ระบุ key ที่ค้นหาไม่พบ

HashTable (Chaining)

```python
def _rehash(self) -> None:
    """ขยาย capacity 2 เท่าและ re-insert ทุก entry"""
    old_buckets = self._buckets
    self._capacity *= 2
    self._buckets = [[] for _ in range(self._capacity)]
    self._size = 0
    for bucket in old_buckets:
        for key, value in bucket:
            self.insert(key, value)          # re-insert ทุก entry

def __len__(self) -> int:
    return self._size
```

```python
def __contains__(self, key) -> bool:
    try:
        self.get(key)
        return True
    except KeyError:
        return False

def __str__(self) -> str:
    lines = [f"HashTable(size={self._size}, "
             f"capacity={self._capacity}, "
             f"load={self._load_factor():.2f})"]
    for i, bucket in enumerate(self._buckets):
        if bucket:
            pairs = ", ".join(f"{k!r}:{v!r}" for k, v in bucket)
            lines.append(f"  [{i:2d}] → {pairs}")
    return "\n".join(lines)
```

- _rehash ตั้งค่า _capacity เป็นสองเท่า
  - สร้าง _buckets ใหม่ทั้งหมด รีเซ็ต _size เป็น 0 แล้วจึง re-insert ทุก entry จาก old_buckets ผ่านการเรียก self.insert() ซ้ำ (ซึ่งจะเพิ่ม _size กลับขึ้นมาใหม่ทีละหนึ่งจนครบ)
  - รีเซ็ต _size เป็น 0 ก่อนเสมอ มิฉะนั้นจะนับซ้ำ
- __contains__ ทำให้สามารถใช้ syntax key in hash_table ได้โดยตรง โดยอาศัยการเรียก get() แล้วดักจับ KeyError แปลงเป็น False แทน

ทดสอบการทำงาน

```
ht = HashTable(initial_capacity=4)
words = [("apple", 5), ("banana", 6), ("cherry", 6), ("date", 4),
("elderberry", 10)]
for word, count in words:
    ht.insert(word, count)
    print(f"insert({word!r}) → load={ht._load_factor():.2f}, capacity=
{ht._capacity}")

print()
print(ht)
print()
print(f"get('apple')  = {ht.get('apple')}")
print(f"get('cherry') = {ht.get('cherry')}")
ht.insert("apple", 99)
print(f"after update, get('apple') = {ht.get('apple')}")
ht.delete("date")
print(f"'date' in ht: {'date' in ht}")
try:
    ht.get("mango")
except KeyError as e:
    print(f"KeyError: {e}")
```

```
insert('apple') → load=0.25, capacity=4
insert('banana') → load=0.50, capacity=4
insert('cherry') → load=0.75, capacity=4
insert('date') → load=0.50, capacity=8
insert('elderberry') → load=0.62, capacity=8

HashTable(size=5, capacity=8, load=0.62)
  [ 1] → 'banana':6, 'date':4
  [ 2] → 'cherry':6
  [ 3] → 'elderberry':10
  [ 7] → 'apple':5

get('apple')  = 5
get('cherry') = 6
after update, get('apple') = 99
'date' in ht: False
KeyError: "Key not found: 'mango'"
```

- ตำแหน่ง bucket ที่แสดงในผลลัพธ์ (เช่น [1], [2], [3], [7]) อาจแตกต่างกันไปในการรันแต่ละครั้ง เพราะ Python สุ่ม hash seed ของ string ใหม่ทุกครั้งที่เริ่ม interpreter
- แต่ค่า load factor ในแต่ละขั้นตอน และพฤติกรรมของ get/insert/delete/KeyError จะเหมือนเดิมเสมอทุกครั้งที่รัน

ทดสอบการทำงาน

```
insert('apple') → load=0.25, capacity=4
insert('banana') → load=0.50, capacity=4
insert('cherry') → load=0.75, capacity=4
insert('date') → load=0.50, capacity=8
insert('elderberry') → load=0.62, capacity=8

HashTable(size=5, capacity=8, load=0.62)
  [ 1] → 'banana':6, 'date':4
  [ 2] → 'cherry':6
  [ 3] → 'elderberry':10
  [ 7] → 'apple':5

get('apple')  = 5
get('cherry') = 6
after update, get('apple') = 99
'date' in ht: False
KeyError: "Key not found: 'mango'"
```

- จากผลลัพธ์ มีตัวอย่าง collision ที่เกิดขึ้นจริง นั่นคือ "banana" และ "date" ชนกันที่ bucket เดียวกัน (index 1) — ทั้งสอง key อยู่ร่วมกันได้ในรูปแบบ list [('banana', 6), ('date', 4)] โดยไม่มีปัญหาใดๆ เลย
- หลัง insert "cherry" load factor แตะ 0.75 พอดี แต่ rehash ยังไม่เกิดขึ้นทันที เพราะการตรวจสอบเงื่อนไข if self._load_factor() >= 0.75 อยู่ที่ต้นของการเรียก insert ครั้งถัดไป (คือตอนเรียก insert("date"))
- นี่คือเหตุผลที่ผลลัพธ์แสดง capacity=4 ต่อจาก insert('cherry') แล้วเปลี่ยนเป็น capacity=8 ก็ต่อเมื่อ insert('date') เริ่มทำงานแล้วเท่านั้น

Code ของ คลาส LinearProbingHashTable

คลาส LinearProbingHashTable

```
class LinearProbingHashTable:
    """
    Hash Table ด้วย Linear Probing
    In-place collision resolution -- ไม่ใช้ linked list
    ต้องใช้ Tombstone เมื่อ delete เพื่อไม่ให้ search chain ขาดกลางคัน
    """
    _TOMBSTONE = object()    # sentinel ที่ไม่ซ้ำกับค่าใดๆ ในโลก

    def __init__(self, capacity: int = 8):
        self._capacity = capacity
        self._size = 0
        self._table = [None] * capacity

    def _hash(self, key) -> int:
        return hash(key) % self._capacity
```

```
def insert(self, key, value) -> None:
    idx = self._hash(key)
    while (self._table[idx] is not None and
           self._table[idx] is not self._TOMBSTONE):
        if self._table[idx][0] == key:        # key มีอยู่แล้ว -> อัปเดต
            self._table[idx] = (key, value)
            return
        idx = (idx + 1) % self._capacity      # probe ไปข้างหน้า
    self._table[idx] = (key, value)
    self._size += 1

def get(self, key):
    idx = self._hash(key)
    while self._table[idx] is not None:
        if (self._table[idx] is not self._TOMBSTONE and
                self._table[idx][0] == key):
            return self._table[idx][1]
        idx = (idx + 1) % self._capacity
    raise KeyError(key)
```

คลาส LinearProbingHashTable

```
    def delete(self, key) -> None:
        idx = self._hash(key)
        while self._table[idx] is not None:
            if (self._table[idx] is not self._TOMBSTONE and
                    self._table[idx][0] == key):
                self._table[idx] = self._TOMBSTONE  # ไม่ใส่ None เพราะจะตัด chain
                self._size -= 1
                return
            idx = (idx + 1) % self._capacity
        raise KeyError(key)

    def __str__(self) -> str:
        lines = [f"LinearProbingHashTable(size={self._size}, capacity=
{self._capacity})"]
        for i, slot in enumerate(self._table):
            if slot is None:
                lines.append(f"  [{i:2d}] → None")
            elif slot is self._TOMBSTONE:
                lines.append(f"  [{i:2d}] → TOMBSTONE")
            else:
                lines.append(f"  [{i:2d}] → {slot[0]!r}:{slot[1]!r}")
        return "\n".join(lines)
```

สาธิต Tombstone Bug

ลองจำลองสถานการณ์ที่บังคับให้ "A" และ "B" ชนกันที่ index เดียวกันเสมอ (เพื่อควบคุมการสาธิตให้แน่นอน ไม่ขึ้นกับ hash seed แบบสุ่ม)

```python
class ForcedCollisionTable(LinearProbingHashTable):
    def _hash(self, key):
        forced = {"A": 2, "B": 2}       # บังคับให้ A, B ชนกันที่ index 2 เสมอ
        return forced.get(key, hash(key) % self._capacity)

t = ForcedCollisionTable(capacity=8)
t.insert("A", "value_A")
t.insert("B", "value_B")      # ชนกับ A ที่ index 2 -> ถูก probe ไปที่ index 3
print(t)
print(f"\nget('B') ก่อน delete 'A': {t.get('B')}")

# จำลองบั๊ก: ลบ A โดยใส่ None ตรงๆ (ไม่ใช่ TOMBSTONE)
idx_a = t._hash("A")
t._table[idx_a] = None
t._size -= 1
print(f"\nหลัง 'ลบ A' แบบผิดๆ (ใส่ None แทน Tombstone):")
print(t)

print(f"\nพยายาม get('B'): ", end="")
try:
    print(t.get("B"))
except KeyError as e:
    print(f"KeyError: {e}  <-- บั๊ก! B ยังอยู่จริง แต่หาไม่เจอ")
```

```
LinearProbingHashTable(size=2, capacity=8)
  [ 0] → None
  [ 1] → None
  [ 2] → 'A':'value_A'
  [ 3] → 'B':'value_B'
  [ 4] → None
  [ 5] → None
  [ 6] → None
  [ 7] → None

get('B') ก่อน delete 'A': value_B

หลัง 'ลบ A' แบบผิดๆ (ใส่ None แทน Tombstone):
LinearProbingHashTable(size=1, capacity=8)
  [ 0] → None
  [ 1] → None
  [ 2] → None
  [ 3] → 'B':'value_B'
  [ 4] → None
  [ 5] → None
  [ 6] → None
  [ 7] → None

พยายาม get('B'): KeyError: 'B'  <-- บั๊ก! B ยังอยู่จริง แต่หาไม่เจอ
```

"B" ยังคงอยู่ที่ index 3 อย่างสมบูรณ์ แต่ get("B") กลับหาไม่เจอ
เพราะการค้นหาเริ่มจาก index 2 (ตำแหน่ง hash ของ B) พบ None ทันที และหยุดการค้นหาโดยไม่ไปต่อยัง index 3 เลย

สาธิต Tombstone Bug

ลองเปรียบเทียบกับการใช้ delete() ที่ถูกต้อง (ใส่ Tombstone แทน)

```
t2 = ForcedCollisionTable(capacity=8)
t2.insert("A", "value_A")
t2.insert("B", "value_B")
t2.delete("A")              # ใช้ delete() จริง ซึ่งใส่ TOMBSTONE ไม่ใช่ None
print(f"get('B') หลัง delete('A') ด้วย Tombstone: {t2.get('B')}")
```

```
get('B') หลัง delete('A') ด้วย Tombstone: value_B
```

เมื่อใช้ Tombstone อย่างถูกต้อง get("B") ค้นเจอได้สำเร็จ
เพราะเมื่อพบ Tombstone ที่ index 2 การค้นหาจะไม่หยุดแต่ probe ต่อไปยัง index 3 จนพบ "B" ในที่สุด
นี่คือหลักฐานที่เป็นรูปธรรมที่สุดว่าทำไม Tombstone จึงจำเป็นอย่างยิ่งสำหรับ Linear Probing